



C++: Operator Overloading

IF2210 – Semester II 2025/2026

Sumber: Diktat Bahasa C++ oleh Hans Dulimarta

Modernisasi oleh Satrio Adi Rukmono

Operator Overloading

- Di sesi sebelumnya pernah disebut *function overloading*: fungsi dengan nama sama tapi *signature* berbeda.
 - (Keberadaan ctor `Stack()`, `Stack(int)`, dan `Stack(Stack&)` dalam 1 kelas sudah merupakan contoh *overloading*.)
- *Operator overloading* memungkinkan kita membuat *custom implementation* untuk operator standar seperti `+`, `*`, `<<` untuk objek.
- Di sini kita bukan sekedar belajar fitur Bahasa C++, melainkan bagaimana objek mengekspresikan perilaku tanpa membocorkan representasi.
 - Tujuan: menjaga **enkapsulasi** + **message passing** tetap terlihat di kode.

Masalah desain

- Pendekatan prosedural:

```
Money total = add(price, tax);
```

- Pertanyaan:
 - Siapa memahami aturan penjumlahan uang?
 - Siapa menjaga *invariant*?
 - Mengapa logika berada di luar objek?
- Fungsi mulai “tahu terlalu banyak”.

Mengembalikan Tanggung Jawab ke Objek

```
class Money {  
public:  
    Money add(const Money& other) const;  
};
```

- Interpretasi: Kita meminta objek melakukan pekerjaannya sendiri.
- Objek menjaga:
 - validitas
 - aturan domain
 - representasi internal

Sintaks “Natural”

- Kita ingin menulis seperti bahasa domain:

`price + tax`

bukan:

`add(price, tax)`

atau:

`price.add(tax)`

- Beberapa bahasa OO berusaha menyelaraskan:
 - Bahasa program = bahasa domain.

Apa Itu *Operator Overloading* (Secara Konseptual)

- *Operator overloading* adalah cara objek menerima pesan menggunakan **notasi** operator.
- Tetap menggunakan konsep OO “mengirim pesan ke objek”, bukan “memanggil fungsi global”.
- Ekspresi $a + b$ dibaca sebagai:
 - “a, tolong gabungkan dirimu dengan b dan beritahu saya hasilnya.”bukan:
 - “jalankan operator +.”

Menjaga Enkapsulasi

```
class Money {  
    public:  
        Money(long value);  
  
        Money operator+(const Money& rhs) const {  
            return Money(value() + rhs.value());  
        }  
  
        long value() const;  
  
    private:  
        long cents_;  
};
```

Dunia luar:

- tidak tahu representasi
- tidak bisa melanggar aturan
- hanya bisa mengirim pesan

Kapan *Operator Overloading* Tepat?

- Gunakan jika operasi tersebut:
 - Mendefinisikan makna tipe
 - Alami dalam domain
 - Diharapkan oleh pembaca
- Contoh:
 - Money + Money
 - Vector + Vector
 - Matrix * Matrix
 - Map[indeks]
- Contoh buruk:

```
stack << x; // push x ke stack
```
- Masalah:
 - Tidak ada makna “<<” dalam konsep *stack*
- Tidak bisa ditebak pembaca
- Menyembunyikan niat

Operator harus **memperjelas makna**, bukan menyamarkan.

Catatan: Bentuk Lain yang Mungkin Anda Temui di Dunia Nyata

- Di beberapa kode C++, operator **tidak ditulis sebagai anggota kelas**, melainkan sebagai fungsi bebas (*free function*). Contoh:

```
Money operator+(const Money& a, const Money& b) {  
    return Money(a.value() + b.value());  
}
```

- Ini tetap memungkinkan pemakaian:

```
Money total = price + tax;
```

- Bentuk ini ada karena alasan historis / teknis dalam C++.
- Anda diharapkan untuk tidak kembali ke cara berpikir prosedural ini.

Kaitan ke Polimorfisme

- Kita bisa punya banyak kelas yang sama-sama merepresentasikan uang:

```
PreciseMoney m1{100};  
ConvertedMoney m2{50, "EUR"};
```

- Kode ini tetap bekerja (asalkan ketiga kelas terkait dirancang dan diimplementasi dengan benar):

```
Money total = m1 + m2;
```

- Bagaimana caranya? Akan dipelajari di sesi lain.

Penutup

- *Operator overloading* **bukan untuk:**
 - trik sintaks
 - membuat kode lebih pendek
 - menunjukkan kepintaran C++
- **Heuristik desain:** Tanya sebelum *overload*:
 - Tanpa operasi ini, apakah konsep tipe terasa “rusak”?
 - Jika ya → *overload* masuk akal.
 - Jika tidak → gunakan *method* biasa.